

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA0615	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Soundariya M	Reg. No.:	192524342
Section / Batch:	Btech-AI&DS	Date:	28:02:2026

Instructions to Students

- Answer both questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identifies all key issues.
- Thorough analysis with validated results and performance evaluation

Question Type	Description	Questions	Total Marks
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1 – Q2	100

SECTION A – Industry Problem Solving Task

QUESTION 1:

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

Soundariya M
1925243450A

CO4-AT2-Industry problem solving

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Greedy Technique
50 marks

Q2
Greedy Technique
50 marks

2/2 answered

Q1 Greedy Technique

50 marks

Smart Inventory Replenishment Optimization (Dynamic Programming)

A retail chain must determine optimal inventory replenishment schedules to minimize holding and shortage costs. Analyze how this problem can be formulated using Dynamic Programming. Identify constraints such as demand variability, storage limits, and lead times. Design a solution using recurrence relations for optimal decision-making. Discuss implementation feasibility in supply chain systems. Evaluate algorithm efficiency and scalability. Interpret performance in terms of cost savings and inventory optimization.

Your Code

```
1 # Smart Inventory Replenishment Optimisation (Simplified DP Version)
2
3 def inventory_optimisation(demand, max_capacity, holding_cost, shortage_cost, order_cost):
4     """
5     demand: list of demand values for each period (easy input)
6     max_capacity: maximum inventory that can be stored
7     holding_cost: cost per unit held at end of period
8     shortage_cost: cost per unit of unmet demand
9     order_cost: cost per unit ordered
10    """
11    T = len(demand)
12
13    # dp[t][i] = minimum cost to reach inventory level i at end of period t
14    dp = [[float('inf')] * (max_capacity + 1) for _ in range(T + 1)]
15    dp[0][0] = 0 # Start with 0 inventory and 0 cost
16
17    # To trace back the chosen order quantity
18    choice = [[0] * (max_capacity + 1) for _ in range(T + 1)]
19
20    for t in range(1, T + 1):
21        d = demand[t - 1]
22        for prev_inv in range(max_capacity + 1):
23            if dp[t - 1][prev_inv] == float('inf'):
24                continue
25            for order_qty in range(0, max_capacity + 1):
26                available = prev_inv + order_qty
27                new_inv = available - d
28                shortage = max(0, -new_inv)
29                end_inv = max(0, new_inv)
30
31                if end_inv > max_capacity:
32                    continue
33
34                cost = (order_qty * order_cost) + (end_inv * holding_cost) + (shortage * shortage_cost)
35                total_cost = dp[t - 1][prev_inv] + cost
36
37                if total_cost < dp[t][end_inv]:
38                    dp[t][end_inv] = total_cost
39                    choice[t][end_inv] = order_qty
40
41    # Find best final cost
42    min_cost = min(dp[T])
43    best_inv = dp[T].index(min_cost)
44
```

Input

5 10 5
15
1
5
-

Output

Minimum Total Cost: 40
Optimal Order Schedule: [5, 10, 5]

QUESTION 2:



SIMATS Online Programming Lab
DAA PROGRAMMING

Soundariya M
192524342RA

CO4-AT2-Industry problem solving

Language: Python C C++ Java

QUESTIONS

Q1
Greedy Technique
50 marks

Q2
Greedy Technique
50 marks

2/2 answered

Q2 Greedy Technique

50 marks

Huffman Coding ♦ Data Compression
A video streaming platform compresses data based on symbol frequencies to optimize storage and transmission. Analyze how Huffman coding can be used to generate optimal prefix codes. Design the solution by explaining tree construction and encoding process. Suggest tools for implementing compression. Analyze compression efficiency and time complexity. Interpret the results in terms of bandwidth and storage optimization.

Your Code

```
1 import heapq
2 from collections import Counter
3
4 class Node:
5     def __init__(self, freq, char=None, left=None, right=None):
6         self.freq = freq
7         self.char = char # symbol at leaf nodes
8         self.left = left
9         self.right = right
10
11 # for priority queue: compare by frequency
12 def __lt__(self, other):
13     return self.freq < other.freq
14
15 def build_huffman_tree(text):
16     freq_map = Counter(text)
17
18     heap = []
19     for ch, freq in freq_map.items():
20         heapq.heappush(heap, Node(freq, char=ch))
21
22 # Edge case: if only one unique character
23 if len(heap) == 1:
24     only = heapq.heappop(heap)
25     root = Node(only.freq, left=None, right=None)
26     return root
27
28 while len(heap) > 1:
29     a = heapq.heappop(heap) # smallest
30     b = heapq.heappop(heap) # second smallest
31
32     merged = Node(a.freq + b.freq, left=a, right=b)
33     heapq.heappush(heap, merged)
34
35 return heap[0] # root
36
37 def build_codes(root):
38     codes = {}
39
40     def dfs(node, current_code):
41         if node is None:
42             return
43         # leaf node
44         if node.char is not None:
```

Input

aaabbc

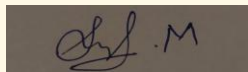
Output

Frequencies: ('a': 3, 'b': 2, 'c': 1)
Huffman Codes: ('a': '0', 'c': '10', 'b': '11')
Encoded Bitstring: 000111110
Decoded Text: aaabbc

Code of Conduct

I, Soundariya M(192524342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Mark Evaluation:

Case Study	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism (5)	Total Marks (Each - 50)
Q1							
Q2							
Overall Total (100)							

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____